CSC240_BST/TreeType.cpp

```
 1
 2  #include "TreeType.h"
 3  #include <iostream>
 4  using namespace std;
 5  struct TreeNode
 6  {
 7    ItemType info;
 8    TreeNode* left;
 9    TreeNode* right;
10  };
11
12  bool TreeType::IsFull() const
13  // Returns true if there is no room for another item
14  //  on the free store; false otherwise.
15  {
16    TreeNode* location;
17    try
18    {
19      location = new TreeNode;
20      delete location;
21      return false;
22    }
23    catch(std::bad_alloc& exception)
24    {
25      return true;
26    }
27  }
28
29  bool TreeType::IsEmpty() const
30  // Returns true if the tree is empty; false otherwise.
31  {
32    return root == NULL;
33  }
34
35  int CountNodes(TreeNode* tree);
36
37  int TreeType::GetLength() const
38  // Calls recursive function CountNodes to count the
39  // nodes in the tree.
40  {
41    return CountNodes(root);
42  }
43
44
45  int CountNodes(TreeNode* tree)
46  // Post: returns the number of nodes in the tree.
47  {
```

```
48    if (tree == NULL)
49      return 0;
50    else
51      return CountNodes(tree->left) + CountNodes(tree->right) + 1;
52  }
53
54  void Retrieve(TreeNode* tree,
55       ItemType& item, bool& found);
56
57  ItemType TreeType::GetItem(ItemType item, bool& found)
58  // Calls recursive function Retrieve to search the tree for item.
59  {
60    Retrieve(root, item, found);
61    return item;
62  }
63
64
65  void Retrieve(TreeNode* tree,
66       ItemType& item, bool& found)
67  // Recursively searches tree for item.
68  // Post: If there is an element someItem whose key matches item's,
69  //       found is true and item is set to a copy of someItem;
70  //       otherwise found is false and item is unchanged.
71  {
72    if (tree == NULL)
73      found = false;                     // item is not found.
74    else if (item < tree->info)
75      Retrieve(tree->left, item, found); // Search left subtree.
76    else if (item > tree->info)
77      Retrieve(tree->right, item, found);// Search right subtree.
78    else
79    {
80      item = tree->info;                 // item is found.
81      found = true;
82     }
83  }
84
85  void Insert(TreeNode*& tree, ItemType item);
86
87  void TreeType::PutItem(ItemType item)
88  // Calls recursive function Insert to insert item into tree.
89  {
90    Insert(root, item);
91  }
92
93
94  void Insert(TreeNode*& tree, ItemType item)
95  // Inserts item into tree.
96  // Post:  item is in tree; search property is maintained.
```

```
 97 {
 98   if (tree == NULL)
 99   {// Insertion place found.
100     tree = new TreeNode;
101     tree->right = NULL;
102     tree->left = NULL;
103     tree->info = item;
104   }
105   else if (item < tree->info)
106     Insert(tree->left, item);    // Insert in left subtree.
107   else
108     Insert(tree->right, item);   // Insert in right subtree.
109 }
110
111 void DeleteNode(TreeNode*& tree);
112
113 void Delete(TreeNode*& tree, ItemType item);
114
115 void TreeType::DeleteItem(ItemType item)
116 // Calls recursive function Delete to delete item from tree.
117 {
118   Delete(root, item);
119 }
120
121
122 void Delete(TreeNode*& tree, ItemType item)
123 // Deletes item from tree.
124 // Post:  item is not in tree.
125 {
126   if (item < tree->info)
127     Delete(tree->left, item);   // Look in left subtree.
128   else if (item > tree->info)
129     Delete(tree->right, item);  // Look in right subtree.
130   else
131     DeleteNode(tree);           // Node found; call DeleteNode.
132 }
133
134 void GetPredecessor(TreeNode* tree, ItemType& data);
135
136 void DeleteNode(TreeNode*& tree)
137 // Deletes the node pointed to by tree.
138 // Post: The user's data in the node pointed to by tree is no
139 //       longer in the tree.  If tree is a leaf node or has only
140 //       non-NULL child pointer the node pointed to by tree is
141 //       deleted; otherwise, the user's data is replaced by its
142 //       logical predecessor and the predecessor's node is deleted.
143 {
144   ItemType data;
145   TreeNode* tempPtr;
```

```
146
147   tempPtr = tree;
148   if (tree->left == NULL)
149   {
150     tree = tree->right;
151     delete tempPtr;
152   }
153   else if (tree->right == NULL)
154   {
155     tree = tree->left;
156     delete tempPtr;
157   }
158   else
159   {
160     GetPredecessor(tree->left, data);
161     tree->info = data;
162     Delete(tree->left, data);  // Delete predecessor node.
163   }
164 }
165
166 void GetPredecessor(TreeNode* tree, ItemType& data)
167 // Sets data to the info member of the right-most node in tree.
168 {
169   while (tree->right != NULL)
170     tree = tree->right;
171   data = tree->info;
172 }
173
174 void PrintTree(TreeNode* tree, std::ostream& outStream)
175 // Prints info member of items in tree in sorted order on outFile.
176 {
177   if (tree != NULL)
178   {
179     PrintTree(tree->left, outStream);   // Print left subtree.
180     outStream << tree->info;
181     PrintTree(tree->right, outStream);  // Print right subtree.
182   }
183   return;
184 }
185
186 void TreeType::Print(std::ostream& outStream) const
187 // Calls recursive function Print to print items in the tree.
188 {
189   PrintTree(root, outStream);
190   outStream << "\n";
191 }
192
193 void TreeType::PrintOrder(OrderType order)
194 // Calls recursive function Print to print items in the tree.
```

```
195 {
196     switch(order){
197         case PRE_ORDER:
198             cout << "PRE: ";
199             break;
200         case POST_ORDER:
201             cout << "POST: ";
202             break;
203         case IN_ORDER:
204             cout << "IN: ";
205             break;
206     }
207     if(root != NULL){
208         bool finished = false;
209         while(!finished){
210             cout << GetNextItem(order, finished);
211         }
212         cout << "\n";
213     }
214 }
215
216 TreeType::TreeType()
217 {
218   root = NULL;
219 }
220
221 void Destroy(TreeNode*& tree);
222
223 TreeType::~TreeType()
224 // Calls recursive function Destroy to destroy the tree.
225 {
226   Destroy(root);
227 }
228
229
230 void Destroy(TreeNode*& tree)
231 // Post: tree is empty; nodes have been deallocated.
232 {
233   if (tree != NULL)
234   {
235     Destroy(tree->left);
236     Destroy(tree->right);
237     delete tree;
238   }
239 }
240
241 void TreeType::MakeEmpty()
242 {
243   Destroy(root);
```

```
244   root = NULL;
245 }
246
247
248 void CopyTree(TreeNode*& copy,
249      const TreeNode* originalTree);
250
251 TreeType::TreeType(const TreeType& originalTree)
252 // Calls recursive function CopyTree to copy originalTree
253 //  into root.
254 {
255   CopyTree(root, originalTree.root);
256 }
257
258 void TreeType::operator=
259      (const TreeType& originalTree)
260 // Calls recursive function CopyTree to copy originalTree
261 // into root.
262 {
263   {
264   if (&originalTree == this)
265     return;             // Ignore assigning self to self
266   Destroy(root);      // Deallocate existing tree nodes
267   CopyTree(root, originalTree.root);
268   }
269
270 }
271 void CopyTree(TreeNode*& copy,
272      const TreeNode* originalTree)
273 // Post: copy is the root of a tree that is a duplicate
274 //       of originalTree.
275 {
276   if (originalTree == NULL)
277     copy = NULL;
278   else
279   {
280     copy = new TreeNode;
281     copy->info = originalTree->info;
282     CopyTree(copy->left, originalTree->left);
283     CopyTree(copy->right, originalTree->right);
284   }
285 }
286 // Function prototypes for auxiliary functions.
287
288 void PreOrder(TreeNode*, QueType&);
289 // Enqueues tree items in preorder.
290
291
292 void InOrder(TreeNode*, QueType&);
```

```
293 // Enqueues tree items in inorder.
294
295
296 void PostOrder(TreeNode*, QueType&);
297 // Enqueues tree items in postorder.
298
299
300 void TreeType::ResetTree(OrderType order)
301 // Calls function to create a queue of the tree elements in
302 // the desired order.
303 {
304   switch (order)
305   {
306     case PRE_ORDER : PreOrder(root, preQue);
307                      break;
308     case IN_ORDER  : InOrder(root, inQue);
309                      break;
310     case POST_ORDER: PostOrder(root, postQue);
311                      break;
312   }
313 }
314
315
316 void PreOrder(TreeNode* tree,
317      QueType& preQue)
318 // Post: preQue contains the tree items in preorder.
319 {
320   if (tree != NULL)
321   {
322     preQue.Enqueue(tree->info);
323     PreOrder(tree->left, preQue);
324     PreOrder(tree->right, preQue);
325   }
326 }
327
328
329 void InOrder(TreeNode* tree,
330      QueType& inQue)
331 // Post: inQue contains the tree items in inorder.
332 {
333   if (tree != NULL)
334   {
335     InOrder(tree->left, inQue);
336     inQue.Enqueue(tree->info);
337     InOrder(tree->right, inQue);
338   }
339 }
340
341
```

```
342 void PostOrder(TreeNode* tree, QueType& postQue)
343 // Post: postQue contains the tree items in postorder.
344 {
345   if (tree != NULL)
346   {
347     PostOrder(tree->left, postQue);
348     PostOrder(tree->right, postQue);
349     postQue.Enqueue(tree->info);
350   }
351 }
352
353
354 ItemType TreeType::GetNextItem(OrderType order, bool& finished)
355 // Returns the next item in the desired order.
356 // Post: For the desired order, item is the next item in the queue.
357 //       If item is the last one in the queue, finished is true;
358 //       otherwise finished is false.
359 {
360   finished = false;
361   ItemType item;
362   switch (order)
363   {
364     case PRE_ORDER  : preQue.Dequeue(item);
365                       if (preQue.IsEmpty())
366                         finished = true;
367                       break;
368     case IN_ORDER   : inQue.Dequeue(item);
369                       if (inQue.IsEmpty())
370                         finished = true;
371                       break;
372     case  POST_ORDER: postQue.Dequeue(item);
373                       if (postQue.IsEmpty())
374                         finished = true;
375                       break;
376   }
377   return item;
378 }
379
380 void TreeType::checkSizes(std::ostream& outStream) const{
381     outStream << "\n" << sizeof(root) << " " << sizeof(*root);
382
383 }
384
385
386
387
```